Research Proposal

Deep Concept Model (DCM):
A Formal Concept Algebra for Compositional Reasoning Beyond Language

Towards a Computable Ontology with Verified Concept Generation

Hoyeon Luke Jang

March 2026

Draft for academic review and collaboration enquiry

Abstract

This proposal presents the Deep Concept Model (DCM), a research programme that aims to construct a formal concept algebra grounded in type theory, instantiated as a computable ontology over a mathematical domain, with a verified mechanism for generating new concepts from primitive elements. The central thesis is that meaningful reasoning can and should occur at a level of abstraction above natural language—at the level of *concepts* themselves—and that a system capable of composing, verifying, and registering new concepts from axiomatic primitives would represent a foundational advance in neuro-symbolic AI and knowledge representation.

The project draws on three convergent insights: the observation, articulated by Andrej Karpathy, that future AI systems may develop forms of reasoning that transcend human linguistic comprehension; the Wittgensteinian thesis that the boundaries of language constrain the boundaries of thought; and the creative-combinatorial principle—drawn from research on creativity and innovation—that novel ideas emerge through the structured combination of previously unconnected concepts. DCM proposes that if axioms constitute the atomic layer of conceptual structure, then the systematic combination and verification of axioms can simulate the full generative process by which new knowledge is produced.

This proposal scopes the work to Layers 0 through 2 of the DCM architecture: philosophical foundation, ontological structure, and concept generation with verification. The target domain is formal mathematics, where concepts are compositional by design, verification reduces to proof-checking, and an extensive formal library (Lean 4's Mathlib) already provides the infrastructure on which to build. Future work, outlined briefly, extends the framework to model training and curriculum-based reasoning over the generated ontology.

1. Introduction and Motivation

1.1 The Language Bottleneck in Machine Reasoning

Large language models have achieved remarkable performance across a wide range of cognitive tasks, from translation and summarisation to mathematical problem-solving and code generation. Yet their underlying mechanism remains fundamentally linguistic: they predict the next token in a

sequence. This architecture inherits a constraint that may prove to be a ceiling on machine reasoning—the constraint of language itself.

Ludwig Wittgenstein, in the *Tractatus Logico-Philosophicus* (1921), proposed that "the limits of my language mean the limits of my world." This assertion—that what can be thought is bounded by what can be expressed—has a direct analogue in modern AI. A model that reasons exclusively in linguistic tokens is, by construction, unable to represent or manipulate structures that exist beyond the expressive capacity of language. If there are forms of reasoning, inference, or abstraction that do not reduce to sequences of words, then token-level models are structurally blind to them.

This is not merely a philosophical concern. In a lecture titled *Deep Dive into LLMs like ChatGPT*, Andrej Karpathy—founding member of OpenAI and former Senior Director of AI at Tesla—drew a striking parallel between the future trajectory of large language models and the behaviour observed in AlphaGo's reinforcement learning. AlphaGo's celebrated Move 37 against Lee Sedol was a move that no human Go player would have considered, yet it emerged from a process of reinforcement learning that optimised for winning rather than for human interpretability. Karpathy observed that LLMs trained with reinforcement learning may similarly develop reasoning pathways that are effective but fundamentally alien to human cognition—reasoning that we cannot comprehend because it operates outside the structures of human language and human conceptual intuition.

This observation suggests a profound opportunity. If AI systems are already developing internal representations that do not map cleanly onto linguistic categories, then the next frontier is not to constrain these systems to think in our language, but to build systems that are *designed* to reason beyond it—at the level of concepts themselves.

1.2 Creativity as Conceptual Combination

A second line of inspiration for this work comes from research on the nature of creativity. In the study of creative cognition and innovation, a recurring finding is that genuinely novel ideas arise not from isolated insight but from the *structured combination of previously unconnected concepts*. Steve Jobs famously combined the computational power of the personal computer with the aesthetic principles of typography and graphic design—two domains that, at the time, were

considered unrelated—to produce the Macintosh and, later, the iPhone. What distinguished this as a creative act was not the invention of either component, but the recognition that their combination yielded something qualitatively new.

This principle—that creativity is, in essence, a combinatorial process operating over a space of concepts—has been articulated in various forms across the creativity research literature. It suggests that the generative process by which new knowledge is produced can be modelled as a systematic operation: take existing concepts, combine them through well-defined operations, evaluate whether the result is meaningful, and if so, register it as a new concept available for further combination. This is precisely the mechanism that DCM proposes to formalise and automate.

1.3 From Intuition to Formal System

The Deep Concept Model begins with the premise that *concepts* occupy a level of abstraction above language. We think in concepts, but we identify and communicate them through words. The word is the interface; the concept is the substance. If this is true, then a system that operates directly on concepts—composing them, verifying them, and generating new ones—would be operating at a more fundamental level than any language model.

The core architectural intuition of DCM is as follows. There exist primitive concepts—which we term *axioms*—that cannot be decomposed into simpler conceptual parts. These axioms constitute the atomic layer of the conceptual ontology. Every higher-order concept is, in principle, a composition of two or more axioms or previously composed concepts, combined through a set of well-defined operations and subjected to verification. The iterative application of this combine-verify-register loop generates an expanding ontology of validated concepts: a computational representation of the structure of knowledge itself.

This proposal does not claim that all human knowledge reduces to axiomatic composition—that is a strong philosophical commitment that is testable in narrow domains but difficult to validate in general. What it claims is that for *formal mathematics*, this framework is not only plausible but already implicit in the structure of modern proof assistants. Every theorem in Lean 4's Mathlib library is, by construction, a composition of definitions, lemmas, and axioms. DCM proposes to make this compositional structure explicit, computable, and generative.

2. Background and Related Work

2.1 Meta's Large Concept Model (LCM)

The most direct predecessor to DCM is Meta FAIR's Large Concept Model (LCM), published in December 2024. LCM replaces token-level autoregression with sentence-level autoregression: input text is segmented into sentences, each sentence is encoded into a fixed-size vector in Meta's SONAR embedding space (which supports over 200 languages and speech modalities), and the model predicts the next sentence embedding. The architecture explored three approaches: MSE regression, diffusion-based generation, and quantised SONAR embeddings. The paper's smaller exploratory models used about 1.3 trillion training tokens, and the scaled variant was a 7-billion-parameter diffusion-based Two-Tower architecture trained on about 2.7 trillion tokens.

LCM demonstrated strong zero-shot multilingual generalisation on summarisation tasks, but it did not achieve the philosophical promise implied by its name. Its "concepts" are sentence embeddings—compressed representations of linguistic strings—not independent conceptual structures. The concept space is not compositional: there is no mechanism for building complex concepts from simpler ones. And the SONAR embedding space was designed for cross-lingual translation, not for preserving logical structure, compositionality, or inferential relationships. In short, LCM advanced the granularity of processing from tokens to sentences, but it did not advance the *level of abstraction* from language to concepts.

DCM takes the insight of LCM—that reasoning should operate above the token level—and pushes it further. Where LCM equates a concept with a sentence, DCM defines concepts from first principles as compositional structures built from axiomatic primitives, with verification grounded in formal proof.

2.2 Conceptual Spaces and Concept Algebras

Peter Gärdenfors' framework of *Conceptual Spaces* (2000) provides the most developed formal theory of concepts as geometric objects. In this framework, a concept is a convex region in a multi-dimensional quality space, where each dimension represents a perceivable or measurable quality. Similarity between concepts is geometric distance, and composition occurs through intersection, union, or projection of regions. This framework is powerful for perceptual and graded concepts

but becomes less natural for abstract mathematical concepts where quality dimensions are not obvious.

In a complementary direction, Elmoznino et al. (2024) proposed a complexity-based theory of compositionality grounded in algorithmic information theory: a representation is compositional when it is highly expressive but can be described as a simple function of discrete parts. This provides a measurable criterion for compositionality—a way to evaluate whether a given concept representation actually composes in the way claimed—but it is a property of representations, not a construction method.

DCM draws on both of these traditions: Gärdenfors' geometric intuition informs the embedding and clustering of the ontology, while Elmoznino et al.'s compression-based metric provides an evaluation criterion for the compositionality of generated concepts.

2.3 Type Theory and Formal Mathematics

In dependent type theory—as implemented in proof assistants such as Lean 4, Coq, and Isabelle—propositions are types, proofs are programs, and concepts can be identified with types. The concept "prime number" is a dependent type: the type of natural numbers n such that n has exactly two divisors. Composition of concepts is type construction: product types, function types, dependent pairs, and inductive definitions. Verification of concepts is type-checking, which is decidable.

Lean 4's Mathlib library, with over 120,000 definitions and theorems as of early 2025, already constitutes a vast formally verified ontology of mathematics. The LeanDojo project (Yang et al., 2023) demonstrated that the dependency graph of Mathlib can be extracted and used for premise selection in neural theorem proving. DCM proposes to build on this infrastructure: not to replace Mathlib, but to extract, curate, and augment its structure into an explicit compositional ontology with typed relationships and verified concept generation.

2.4 Library Learning and Concept Discovery

The concept generation mechanism proposed by DCM has a direct analogue in program synthesis. DreamCoder (Ellis et al., 2021) alternates between a "wake" phase (solving problems using a current library of program primitives) and a "sleep" phase (abstracting common patterns from solved problems into new library routines). The sleep phase—discovering reusable abstractions

and registering them for future use—is essentially the combine-verify-register loop that DCM proposes for concept generation.

In the domain of AI for mathematics, recent advances have demonstrated the power of combining neural methods with formal verification. DeepMind's AlphaProof paired a language model with a formal theorem prover to achieve silver-medal performance at the 2024 International Mathematical Olympiad. The LEGO-Prover system (Wang et al., 2024) demonstrated modular proof construction from reusable lemma blocks—a direct structural analogue to building reasoning from compositional concept primitives.

2.5 Ontology Engineering

Formal ontology—the specification of what exists and how things relate—is a mature discipline in computer science. The Web Ontology Language (OWL) and its underlying description logics provide formal languages for defining concepts and relationships with decidable reasoning services. Large-scale knowledge graphs such as Wikidata and ConceptNet encode millions of concepts with typed relationships. The OLLM system (Lo et al., NeurIPS 2024) demonstrated end-to-end ontology learning from text using fine-tuned language models.

For mathematical knowledge specifically, the Cyc project's three decades of manual knowledge engineering offer both inspiration and caution: hand-crafting a complete ontology does not scale. This motivates DCM's approach of building on the already-formalised structure of Mathlib, augmenting it with compositional and hierarchical relationship annotations rather than constructing the ontology from scratch.

3. Theoretical Framework: What Is a Concept?

3.1 The Type-Theoretic Definition

DCM commits to the following formal definition: *a concept is a type in a dependent type theory*. Axioms are the foundational types and definitions from which all other types are constructed. Composition is type construction. Verification is type-checking.

This commitment is motivated by three considerations. First, it gives an immediate, working definition: every definition and theorem in Lean 4's Mathlib is already a type, and the composition

operators of type theory (product types, function types, dependent types, inductive types, predicate restriction) are already the operations by which mathematical concepts combine. Second, verification is built in: Lean's type checker determines whether a type expression is well-formed and, with additional effort, whether it is inhabited (non-empty). Third, the entire Mathlib library becomes the starting ontology—a vast, formally verified body of mathematical knowledge that does not need to be reconstructed from scratch.

3.2 Axioms: The Atomic Layer

In the type-theoretic framing, axioms operate at two levels. At the foundation level, Lean 4's type theory rests on a small number of axioms: propositional extensionality, the quotient construction, and the axiom of choice. These are truly primitive and non-derivable. At the domain level, the definitions and basic lemmas of a specific area of mathematics serve as "axioms" within their domain—they are derived from the foundation but function as primitives for local reasoning. For example, in elementary number theory, the definition of primality, divisibility, and the Euclidean algorithm are domain-level primitives.

DCM treats domain-level axioms as the operative atomic layer. The choice of which concepts to treat as primitive is a design decision with significant implications for the size and depth of the ontology. This proposal recommends selecting primitives at the level of standard undergraduate mathematical definitions: concrete enough to be meaningful, abstract enough to be combinatorially productive.

3.3 Composition: How Concepts Combine

A critical insight of the type-theoretic framework is that composition is not a single operation but a family of operations, each appropriate for different kinds of conceptual combination:

- Product types ($A \times B$): combining two concepts conjunctively. The concept of a "prime ideal" involves the conjunction of "prime" and "ideal" with structural constraints.
- Function types ($A \to B$): concepts that are transformations. The "derivative" is a function from differentiable functions to functions.
- Dependent types ($\Sigma\ x{:}A, B(x)$): concepts whose components are interrelated. A group together with its identity element forms a dependent pair.

- Inductive types: concepts defined by their constructors, such as the natural numbers defined by zero and successor.
- Predicate restriction: the subset of a type satisfying a property. “Even number” is the natural numbers restricted by divisibility by two.

The principle of compositionality, as formalised by Montague and elaborated by subsequent formal semantics, requires a homomorphism between the syntax of combination (how compositions are specified) and the semantics (what the composition means). In the type-theoretic setting, both are already defined: the syntax is type expressions, and the semantics is their interpretation in type theory.

4. Proposed Methodology

4.1 Layer 0: Philosophical Foundation

The first phase of the project establishes the formal definition of “concept” that all subsequent work depends on. The deliverable is a position paper committing to the type-theoretic framing, justifying the choice against alternatives (conceptual spaces, description logics, algorithmic information theory), and specifying the composition operators and their domains of applicability. This phase also includes selecting a target domain—elementary number theory—and curating a list of 20–50 primitive concepts that serve as the axiomatic base.

4.2 Layer 1: Ontological Structure

The second phase constructs the ontology. The methodology involves three stages:

Extraction. Using Lean’s metaprogramming facilities and the LeanDojo extraction tools, the dependency subgraph of a well-defined mathematical result—the Fundamental Theorem of Arithmetic (every natural number greater than 1 is uniquely expressible as a product of primes)—is extracted from Mathlib. This yields a naturally bounded subgraph of approximately 50–100 concepts.

Curation and annotation. The raw dependency graph captures “uses” relationships but not hierarchical or compositional structure. This phase augments the graph with typed relationships: *is-a* (subsumption), *depends-on* (prerequisite), *composed-from* (constructive origin), and *maps-to*

(functional relationships). The representation is a labelled property graph (NetworkX), maintaining bidirectional links to the original Lean definitions for formal verification.

Embedding and clustering. The annotated ontology is embedded in hyperbolic space using Poincaré embeddings (Nickel & Kiela, 2017), which are particularly suited to hierarchical structures because hyperbolic geometry naturally accommodates tree-like topologies with exponentially growing neighbourhoods. Community detection algorithms (Louvain, spectral clustering) are applied to identify conceptual "clusters"—coherent subdomains that correspond to mathematical subfields. This provides both a quantitative structure and an intuitive visualisation of the ontology.

4.3 Layer 2: Concept Generation and Verification

The third and central phase implements the generative mechanism. The combine-verify-register pipeline operates as follows:

Step 1: Component selection. Two or more existing concepts are selected from the ontology. Selection strategies include random sampling (baseline), similarity-based selection (combining nearby concepts), analogy-based selection (combining concepts that play structurally similar roles in different subdomains), and gap-filling (combining concepts that "should" be connected in the ontology but are not).

Step 2: Composition. A composition operator is applied to the selected components. Given the type-theoretic framework, this means constructing a new type expression from the component types. Multiple operators may be attempted for the same pair of components.

Step 3: Verification. Verification proceeds at four levels of increasing stringency:

1. Well-formedness: Is the type expression syntactically valid? This is decided by Lean's type checker in milliseconds.
2. Inhabitedness: Is the type non-empty—does at least one object satisfying the concept exist? This is semi-decidable; the system attempts to construct a witness using Lean's decision tactics.

3. Non-triviality: Is the concept distinct from existing concepts in the ontology? This requires checking logical equivalence against existing types.
4. Interestingness: Does the concept have non-trivial mathematical value? Proxy metrics include proof-shortening (does the new concept simplify existing proofs?), structural bridging (does it connect previously distant parts of the ontology?), and embedding novelty (is it distant from existing concepts in the hyperbolic embedding space?).

Step 4: Registration. Concepts that pass verification are added to the ontology as new nodes, with edges to their components (*composed-from*), hierarchical position (*is-a*), and dependencies (*depends-on*). The new concept receives a deterministic identifier (hash-based or structural) and is available as input for subsequent rounds of generation.

Addressing Combinatorial Explosion

With N concepts and k composition operators, exhaustive binary composition generates $k \times N^2$ candidates per generation step—exponential with recursive application. DCM employs four pruning strategies to manage this: *type-directed search* (restricting compositions to type-compatible pairs, naturally enforced by Lean's type system), *analogy-driven generation* (using structural parallels between domains to guide combination), *curriculum-guided generation* (composing in order of depth, propagating only verified concepts to the next level), and *neural-guided search* (training a lightweight model to predict which compositions are likely to be interesting, following the heuristic-search paradigm of AlphaProof and similar systems).

Asynchronous Generation

A novel architectural feature of DCM is the proposal for asynchronous concept generation: the combine-verify-register loop runs as a continuous background process, independently expanding the ontology regardless of any downstream reasoning task. This is analogous to experience replay buffers in reinforcement learning, to dreaming in world models, and most directly to the "sleep" phase of DreamCoder's library learning. The ontology grows over time, accumulating verified concepts that are available whenever a reasoning task requires them.

5. Expected Contributions

The successful completion of Layers 0 through 2 would yield the following contributions:

1. **A formal concept algebra for mathematics.** A rigorous definition of concepts as types, axioms as foundational definitions, and composition as type construction, with a justified commitment to the type-theoretic framing over alternatives.
2. **A computable ontology of a mathematical domain.** An annotated, embedded, and clustered ontology derived from Mathlib, with typed relationships that go beyond raw dependency information.
3. **A verified concept generation pipeline.** A working system that composes new mathematical concepts from primitives and verifies them through Lean's type checker, with empirical results on the proportion of generated concepts that are well-formed, inhabited, non-trivial, and interesting.
4. **Empirical assessment of automated concept generation.** The first systematic evaluation of whether machine-generated mathematical concepts, produced through compositional combination of primitives, yield non-trivial and mathematically meaningful results.

These contributions are independently publishable at venues such as AAAI, IJCAI, NeurIPS, or specialised workshops on neuro-symbolic AI, knowledge representation, and AI for mathematics.

6. Future Directions: Layers 3 and 4

While the present proposal focuses on the foundational layers of concept definition, ontology construction, and concept generation, the full DCM programme envisions two additional layers that build on these foundations. These are outlined here as future work contingent on the empirical results of Layers 0–2.

6.1 Layer 3: Model Training and Architecture

Once a verified and growing concept ontology exists, the next step is to train a model that reasons natively over this structure. Three architectural paths are under consideration. The first is a *concept augmentation module*—a concept encoder, reasoner, and decoder that interfaces with an existing LLM, handling structured reasoning in concept space while the language model handles natural language input and output. The second is a *fully novel architecture* purpose-built for concept manipulation, potentially leveraging graph neural networks or hyperbolic neural networks. The

third is *concept-aware pre-training*: retaining the transformer architecture but replacing next-token prediction with next-concept prediction over the ontology. The architectural choice will be informed by the structural properties of the ontology produced in Layer 2.

6.2 Layer 4: Curriculum Traversal and Evaluation

The concept ontology, once constructed, defines a natural curriculum for reasoning. Given a target problem, the system would identify the relevant region of the ontology and construct an optimal traversal—a sequence of concepts, ordered by dependency, that constitutes the reasoning pathway to a solution. This curriculum-based approach draws on formal mathematics statement curriculum learning (Polu et al., 2023) and prerequisite chain learning from intelligent tutoring systems. Evaluation would target established mathematical benchmarks (miniF2F, PutnamBench) to provide comparable baselines against existing theorem-proving systems.

7. Key Milestones

The project is organised into sequential phases, each with concrete deliverables. Timelines remain tentative and will be refined as the work progresses.

Phase 1: Foundation

- Formal definition of "concept" in the type-theoretic framing; position paper justifying the choice against alternatives.
- Annotated reading notes on foundational references (Gärdenfors, Elmoznino et al., LCM paper, Lean 4 documentation).
- Curated list of 20–50 primitive concepts for the target domain (elementary number theory), with justification for atomicity.
- Working proficiency in Lean 4 and ability to navigate Mathlib's structure.

Phase 2: Ontology Construction

- Extracted dependency subgraph of the Fundamental Theorem of Arithmetic from Mathlib (~50–100 nodes).
- Augmented ontology graph with typed relationships (is-a, depends-on, composed-from, maps-to) and formal schema specification.

- Poincaré embeddings of the concept graph with community detection and visualisation.

Phase 3: Concept Generation and Evaluation

- Working generation pipeline that composes new types from existing Lean definitions and verifies well-formedness and inhabitedness.
- Non-triviality checking against existing ontology entries.
- At least one implemented interestingness metric (proof shortening, ontology bridging, or embedding novelty).
- First empirical results: statistics on generated concepts (well-formed, inhabited, non-trivial, interesting) from the FTA subgraph.
- Research paper draft suitable for submission to AAAI, IJCAI, NeurIPS, or specialised workshops.

8. Risks and Mitigations

- **Trivial concept generation.** The generated concepts may be well-formed and non-trivial but mathematically uninteresting. Mitigation: develop strong interestingness metrics (proof shortening, ontology bridging, embedding novelty) and prioritise analogy-driven generation over exhaustive search.
- **Lean integration complexity.** Metaprogramming in Lean 4 has a steep learning curve. Mitigation: begin with LeanDojo's existing extraction tools and build custom Lean code only when necessary.
- **Manual annotation burden.** Adding relationship types beyond raw dependency may require significant manual effort. Mitigation: auto-extract where possible and manually annotate only a small evaluation set.
- **Scalability beyond toy domain.** The proof of concept targets 50–200 concepts; scaling to thousands introduces computational and ontological challenges. Mitigation: design the pipeline with scalability in mind from the outset and discuss scaling properties explicitly in the analysis.

9. References

Baader, F., Calvanese, D., McGuinness, D., Nardi, D., & Patel-Schneider, P. (2007). The Description Logic Handbook. Cambridge University Press.

Elmoznino, E., Jiralerspong, T., Bengio, Y., & Lajoie, G. (2024). "A Complexity-Based Theory of Compositionality." arXiv:2410.14817.

Ellis, K. et al. (2021). "DreamCoder: Bootstrapping Inductive Program Synthesis with Wake-Sleep Library Learning." Proceedings of PLDI 2021.

Gärdenfors, P. (2000). Conceptual Spaces: The Geometry of Thought. MIT Press.

Gärdenfors, P. (2014). The Geometry of Meaning. MIT Press.

Guarino, N., Oberle, D., & Staab, S. (2009). "What Is an Ontology?" In Handbook on Ontologies. Springer.

Karpathy, A. (2025). "Deep Dive into LLMs like ChatGPT." YouTube.

LCM Team, Meta FAIR. (2024). "Large Concept Models: Language Modeling in a Sentence Representation Space." arXiv:2412.08821.

Lo, A., Jiang, A.Q., Li, W., & Jamnik, M. (2024). "OLLM: End-to-End Ontology Learning with Large Language Models." NeurIPS 2024.

Martin-Löf, P. (1984). Intuitionistic Type Theory. Bibliopolis.

Nickel, M. & Kiela, D. (2017). "Poincaré Embeddings for Learning Hierarchical Representations." NeurIPS 2017.

Polu, S. et al. (2023). "Formal Mathematics Statement Curriculum Learning." ICLR 2023.

Univalent Foundations Program. (2013). Homotopy Type Theory.

Wang, H. et al. (2024). "LEGO-Prover: Neural Theorem Proving with Growing Libraries." ICLR 2024.

Wittgenstein, L. (1921). Tractatus Logico-Philosophicus.

Yang, K. et al. (2023). "LeanDojo: Theorem Proving with Retrieval-Augmented Language Models." NeurIPS 2023.

Zelikman, E. et al. (2024). "Quiet-STaR: Language Models Can Teach Themselves to Think Before Speaking." arXiv:2403.09629.